

KQL Practice Notes

Practice queries written against the public `StormEvents` sample dataset (`help.Samples` database, Azure Data Explorer sandbox: `dataexplorer.azure.com/clusters/help/databases/Samples`) while learning KQL fundamentals for a Detection Engineering path.

Operators covered

`where` · `project` · `extend` · `summarize` · `count()` · `sum()` · `avg()` · `count_distinct()` · `distinct` · `sort` · `by` · `take` · `top` · `iff()` · `case()` · `monthofyear()` · `contains`

1. Filter rows by a column value

Q: Show only events that happened in Florida.

```
StormEvents
| where State == "FLORIDA"
```

`where` filters rows. `==` is used for comparison (not `=`, which is assignment). String matching is case-sensitive by default.

2. Choose specific columns

Q: From the Florida events, show only EventType, StartTime, and DamageProperty.

```
StormEvents
| where State == "FLORIDA"
| project EventType, StartTime, DamageProperty
```

`project` selects which columns appear in the output, dropping the rest.

3. Most frequent category overall

Q: Which EventType occurred the most across the whole table?

```
StormEvents
| summarize EventCount = count() by EventType
| sort by EventCount desc
| take 1
```

`summarize ... by` groups rows and aggregates within each group. `count()` counts rows per group. Naming the result (`EventCount = count()`) is optional but makes the output readable — without a name, KQL auto-generates `count_`.

4. Sum a numeric column, not row count

Q: Top 5 states by total property damage.

```
StormEvents
| summarize TotalDamage = sum(DamageProperty) by
State
| sort by TotalDamage desc
| take 5
```

Key distinction: `count()` counts rows; `sum(Column)` adds up a numeric column's values. Using `count()` when the question asks for a total amount is a common mix-up.

5. Count distinct values per group

Q: How many unique event types occurred in each state?

```
StormEvents
| summarize UniqueEventTypes =
count_distinct(EventType) by State
| sort by UniqueEventTypes desc
```

`count_distinct()` counts unique values of a column within each group, instead of every row.

6. Create a new column from a condition

Q: Flag each event as "Severe" or "Minor" based on damage amount.

```
StormEvents
| extend RiskLevel = iff(DamageProperty >
```

```
100000, "Severe", "Minor")
| project EventType, DamageProperty, RiskLevel
```

`extend` adds a new computed column without removing existing ones. `iff(condition, if_true, if_false)` is a two-way branch — the building block of simple detection logic ("if this pattern, flag it").

7. Combine conditions with OR

Q: Show events where there were direct injuries OR direct deaths.

```
StormEvents
| where InjuriesDirect > 0 or DeathsDirect > 0
```

`or` means either condition can be true. `and` would require both — a much narrower (and here, wrong) result for this question.

8. Group by a derived time part

Q: Count events per calendar month.

```
StormEvents
| extend month = monthofyear(StartTime)
| summarize EventCount = count() by month
| sort by month asc
```

`monthofyear()` extracts the month number (1–12) from a datetime column. Grouping by a derived column works the same as grouping by a real one.

Bonus — showing month names instead of numbers:

```
StormEvents
| extend month = monthofyear(StartTime)
| extend MonthName = case(
    month == 1, "January",
    month == 2, "February",
    month == 3, "March",
    month == 4, "April",
    month == 5, "May",
    month == 6, "June",
    month == 7, "July",
    month == 8, "August",
    month == 9, "September",
    month == 10, "October",
    month == 11, "November",
    month == 12, "December",
    "Unknown"
)
| summarize EventCount = count() by MonthName
```

`case()` checks multiple conditions in order and returns the value for the first match. The last argument is the default if nothing matches.

9. Filter + project together

Q: Show Tornado events with damage over 500,000 — only State, EventType, DamageProperty.

```
StormEvents
| where EventType == "Tornado" and
DamageProperty > 500000
| project State, EventType, DamageProperty
```

Note: `State` holds state names (e.g. `"TEXAS"`); event categories like `"Tornado"` live in `EventType`. Mixing these up gives an empty or wrong result, not an error.

10. Average per group

Q: Average property damage per event type, highest first.

```
StormEvents
| summarize AvgDamage = avg(DamageProperty) by
EventType
| sort by AvgDamage desc
```

`avg()` — another aggregate function alongside `count()` and `sum()`.

11. Total row count

Q: How many total rows are in the table?

```
StormEvents
| count
```

`count` with no `by` clause collapses the whole table into a single number.

12. Multi-way categorization

Q: Categorize each row as None / Low / High based on damage amount.

```
StormEvents
| extend DamageLevel = case(
    DamageProperty == 0, "None",
    DamageProperty <= 50000, "Low",
    "High"
)
```

`case()` again — this time for three outcomes instead of two, which is where `iff()` stops being enough.

13. Group by two columns at once

Q: Count events by State and EventType combination.

```
StormEvents
| summarize count() by State, EventType
```

`by` accepts multiple columns — the result has one row per unique combination, giving a more detailed breakdown than grouping by a single column.

14. Text search + unique values

Q: Show unique EventType values that contain "Wind".

```
StormEvents
| where EventType contains "Wind"
| distinct EventType
```

`contains` does a substring text search. `distinct` (used alone, without `summarize`) removes duplicate rows for the given column(s).

15. Sum + ranked top-N

Q: Top 3 states by total direct injuries.

```
StormEvents
| summarize TotalInjuries = sum(InjuriesDirect)
by State
| top 3 by TotalInjuries desc
```

`top N by Column desc` is `sort by ... desc | take N` combined into one step.

Key takeaways

- `count()` = number of rows. `sum(Column)` = total of a numeric column. These are not interchangeable.
- Column and table names are case-sensitive (`StormEvents`, not `Storm_Events`; `StartTime`, not `Starttime`).
- `==` compares; `=` assigns. Using `=` in a `where` clause throws a "Filter expression should be Boolean" error.

- `iff()` for two outcomes, `case()` for three or more.
- `by` in `summarize` decides *what* the data is grouped by; the name before `=` is only a display label for the result column.